

系统开发工具基础·实验报告（第 1—4 题）

姓名：李云龙 学号：25030021043

2026 年 8 月 24 日

姓名	李云龙
学号	25030021043
实验日期	2026 年 8 月 24 日
实验环境	Windows 11 + WSL2 (Ubuntu); bash + GNU coreutils (find / cp / sort / uniq / awk / head 等); git; TeX Live (latexmk、pdflatex); poppler-utils (pdftotext)

说明：以下各题“运行结果”均为在 WSL (Ubuntu 26.04 LTS) 中实际运行对应脚本得到的真实输出（运行目录：`/mnt/e/Dyer/系统开发工具基础`）。各题产物均保存在 E 盘对应目录（q01/、q02/、q03/、q04/）。

1 实验目的

1. 掌握含空格文件名、隐藏文件的操作方法（引号、`find -exec`、`cp --parents`），以及目录/文件权限设置。
2. 掌握 Shell 管道与 `awk`、`sort`、`uniq`、`head` 等文本处理工具，完成日志统计。
3. 掌握 Git 分支与合并操作，能够主动制造、解决并解释一次合并冲突。
4. 掌握 LaTeX 文档编辑（公式、表格、交叉引用）与 `latexmk` 命令行构建 PDF。

2 实验一：含空格文件名的批量整理（q01）

2.1 完整脚本（scripts/q01.sh）

```
#!/bin/bash
set -e
SID=25030021043          # 学号
mkdir -p q01/input/docs q01/input/tmp
cd q01

# 1. 创建文件
printf 'alpha\nbeta\n' > 'input/docs/notes one.txt'
echo 'hidden' > input/docs/.secret.txt
touch input/tmp/empty.txt
printf '2026-08-24 10:00:00 INFO start\n2026-08-24 10:01:00 INFO stop\n' > input/run.log
```

```
# 2. 绝对路径 + 长格式列表 (含隐藏文件)
pwd
ls -la input

# 3. 复制所有 .txt 并保留相对目录结构
mkdir -p "work/$SID"
(cd input && find . -type f -name '*.txt' -exec cp --parents {} "../work/$SID/" \;)

# 4. 权限: 目录 750, 普通文件 640
find "work/$SID" -type d -exec chmod 750 {} +
find "work/$SID" -type f -exec chmod 640 {} +

# 5. inventory.txt: 相对路径 + 字节数
find "work/$SID" -type f -exec wc -c {} \; > inventory.txt

# 验证
ls -laR "work/$SID"    # -a 显示隐藏文件; 目录 drwxr-x---, 文件 -rw-r-----
cat inventory.txt
```

2.2 运行结果

`pwd` (q01 的绝对路径):

```
/mnt/e/Dyer/系统开发工具基础/q01
```

`ls -la input` (长格式, 含隐藏项目; `ls -laR input` 可连带列出子目录内容):

```
total 0
drwxr-xr-x 1 root root 4096 Aug 24 21:04 .
drwxr-xr-x 1 root root 4096 Aug 24 21:04 ..
drwxr-xr-x 1 root root 4096 Aug 24 21:04 docs
-rw-r--r-- 1 root root   61 Aug 24 21:04 run.log
drwxr-xr-x 1 root root 4096 Aug 24 21:04 tmp
```

复制后的结构与权限 (`ls -laR work/25030021043`):

```
work/25030021043/:
total 0
drwxr-x--- 1 root root 4096 Aug 24 21:04 .
drwxr-xr-x 1 root root 4096 Aug 24 21:04 ..
drwxr-x--- 1 root root 4096 Aug 24 21:04 docs
drwxr-x--- 1 root root 4096 Aug 24 21:04 tmp

work/25030021043/docs:
total 0
drwxr-x--- 1 root root 4096 Aug 24 21:04 .
drwxr-x--- 1 root root 4096 Aug 24 21:04 ..
-rw-r----- 1 root root    7 Aug 24 21:04 .secret.txt
-rw-r----- 1 root root   11 Aug 24 21:04 notes one.txt
```

```
work/25030021043/tmp:
total 0
drwxr-x--- 1 root root 4096 Aug 24 21:04 .
drwxr-x--- 1 root root 4096 Aug 24 21:04 ..
-rw-r----- 1 root root    0 Aug 24 21:04 empty.txt
```

inventory.txt (相对路径 + 字节数):

```
7 work/25030021043/docs/.secret.txt
11 work/25030021043/docs/notes one.txt
0 work/25030021043/tmp/empty.txt
```

2.3 要点分析

- **空格处理**: `find -exec` 会为 {} 自动加引号、逐个传参, `cp --parents` 原样保留 `notes one.txt` 中的空格, 不会把文件名拆成两个参数。
- **复制范围**: `.secret.txt` 以 `.txt` 结尾, 属于“所有 `.txt` 文件”, 被一并复制; `run.log` 不是 `.txt`, 不复制; `tmp/empty.txt` 是空文件但后缀符合, 也复制。
- **目录结构**: `find . -type f` 输出 `./docs/...` 形式路径, `cp --parents` 在目标下重建 `docs/`、`tmp/` 子目录, `./` 会被路径解析自动归一化, 不影响结构。
- **权限**: 目录 `750 = rwxr-x---` (所有者全部权限, 组可读可执行, 其他无); 普通文件 `640 = rw-r---` (所有者读写, 组只读, 其他无)。
- **字节数核对**: `alpha\nbeta\n` 为 `5+1+4+1=11` 字节; `hidden\n` 为 `7` 字节; 空文件 `0` 字节。

3 实验二：随机访问日志统计 (q02)

3.1 access.csv

按题目给定内容创建 (1 行表头 + 8 行数据, 共 9 行)。

3.2 analyze.sh (scripts/q02.sh 内生成)

```
#!/bin/bash
# 用法: ./analyze.sh <csv文件>
if [ $# -ne 1 ]; then
    echo "用法: $0 <csv文件>" >&2
    exit 2
fi
csv="$1"

if [ ! -f "$csv" ]; then
    echo "错误: 文件 $csv 不存在" >&2
    exit 1
fi
```

```
# 1) 5xx 数量最多的前 2 个 path (次数降序, 相同按字典序)
tail -n +2 "$csv" | awk -F, '$4 >= 500 && $4 < 600 {print $3}' \
    | sort | uniq -c | sort -k1,1nr -k2,2 | head -n 2

# 2) 全部数据行的平均 latency_ms (表头不计入), 保留两位小数
tail -n +2 "$csv" | awk -F, '{sum += $5; n++} END {printf "平均 latency: %.2f\n", sum/n}'
```

3.3 运行结果

对 access.csv 运行:

```
3 /api/orders
2 /api/users
平均 latency: 193.75
```

(`uniq -c` 输出的次数带右对齐前导空格, 属正常现象)

对不存在的文件运行 (错误进入 `stderr`, 退出码非零):

```
错误: 文件 missing.csv 不存在      (stderr)
exit=1
```

3.4 要点分析

- **管道分解:** `tail -n +2` 跳过表头 -> `awk` 过滤状态码 500-599 并取出第 3 列 `path` -> `sort` 让相同 `path` 相邻 -> `uniq -c` 计数 -> `sort -k1,1nr -k2,2` 按次数降序、次数相同按 `path` 字典序 -> `head -n 2` 取前 2。
- **5xx 计数核对:** `/api/orders` 3 次 (500、502、500)、`/api/users` 2 次 (503、500)、`/login` 1 次 (500), 故前 2 名为 `/api/orders`、`/api/users`。
- **平均延迟核对:** $(120+310+180+90+260+20+420+150)/8 = 1550/8 = 193.75$, `printf "%.2f"` 保留两位小数; 表头被 `tail -n +2` 排除, 不计入。
- **错误处理:** 文件不存在时错误信息用 `>&2` 写入 `stderr`, `exit 1` 返回非零退出码, 满足题目要求。

4 实验三：制造、解决并解释一次合并冲突 (q03)

4.1 操作步骤 (scripts/q03.sh)

1. `git init`, 将默认分支命名为 `main`, 创建 `config.txt` (mode=normal) 并完成初始提交。
2. 新建分支 `feature-a`, 把内容改为 `mode=safe` 并提交。
3. 回到 `main` (此时仍在初始提交), 从初始 `main` 新建分支 `feature-b`, 把内容改为 `mode=fast` 并提交。
4. 回到 `main`, 先 `git merge feature-a` (快进合并, 无冲突)。
5. 再 `git merge feature-b`, 产生冲突。

4.2 冲突原因

feature-a 与 feature-b 都从初始提交（mode=normal）出发，各自把同一文件同一行改成了不同内容（mode=safe / mode=fast）。合并时 Git 找不到共同基线下的唯一答案，无法自动合并，于是产生冲突。冲突现场 config.txt 内容：

```
<<<<<<< HEAD
mode=safe
=====
mode=fast
>>>>>>> feature-b
```

4.3 冲突解决

手工编辑（保留 mode=safe，另加一行 note=reviewed），然后标记已解决并提交，完成本次合并：

```
printf 'mode=safe\nnote=reviewed\n' > config.txt
git add config.txt
git commit -m "merge feature-b: keep mode=safe, add note=reviewed"
```

4.4 验证结果

git status: nothing to commit, working tree clean（工作区干净）。

最终 config.txt：

```
mode=safe
note=reviewed
```

git log --all --graph --decorate --oneline：

```
* 8703ad7 (HEAD -> main) merge feature-b: keep mode=safe, add note=reviewed
|\
| * cac5500 (feature-b) feature-b: set mode=fast
* | e3b1f51 (feature-a) feature-a: set mode=safe
|/
* 733bff0 init: mode=normal
```

4.5 小结

- 合并冲突的本质：双方从同一基线修改了同一区域，Git 无法判断保留哪一份，需要人工决策。
- 解决流程：编辑冲突文件（删除 <<<<<<<, =====, >>>>>>> 标记）-> git add -> git commit；也可用 git checkout --ours/--theirs 直接选取一侧。

5 实验四：修复并构建一页技术说明 (q04)

5.1 任务与完整代码

按题目模板补全内容：加入一个带编号和 label 的公式并在正文中引用 (`\eqref`)；加入一个 2 行 2 列表格，设置 caption 与 label 并在正文中引用 (`\ref`)；最后用 `latexmk -pdf -halt-on-error` 构建 PDF，确认交叉引用不显示 “??”。report.tex 完整内容如下：

```
\documentclass{article}
\usepackage{amsmath}
\begin{document}
\title{Tool Report}
\author{Li Yunlong -- 25030021043}
\maketitle
\section{Result}

The mass--energy equivalence is given by Equation~\eqref{eq:einstein}:
\begin{equation}
    E = mc^2
    \label{eq:einstein}
\end{equation}

The sample data are listed in Table~\ref{tab:sample}.
\begin{table}[h]
    \centering
    \caption{Sample data}
    \label{tab:sample}
    \begin{tabular}{cc}
        \hline
        alpha & 2 \\
        beta & 3 \\
        \hline
    \end{tabular}
\end{table}

\end{document}
```

5.2 构建与验证

```
# 安装依赖（首次需要；poppler-utils 提供 pdftotext 用于验证）
sudo apt update
sudo apt install -y texlive-latex-base texlive-latex-recommended texlive-latex-extra latexmk
poppler-utils

# 构建 PDF (latexmk 自动重跑，保证交叉引用正确)
latexmk -pdf -halt-on-error report.tex
ls -l report.pdf

# 验证：PDF 中不应出现未解析引用 “??”（无输出即通过）
```

```
pdftotext report.pdf - | grep -n '??'
```

5.3 运行结果

- report.pdf 成功生成 (1 页, 约 62 KB), `ls -l report.pdf` 确认存在;
- latexmk 自动重跑 2 次编译, 正文中公式引用 `\eqref{eq:einstein}` 显示为 (1), 表格引用 `\ref{tab:sample}` 显示为 1;
- `pdftotext report.pdf - | grep -n '??'` 无输出, 检查脚本输出 OK: PDF 中没有 ??、OK: 日志中无 undefined references。

5.4 要点分析

- **公式编号与引用:** `\begin{equation} ... \label{eq:einstein} \end{equation}` 自动编号, 正文用 `\eqref{eq:einstein}` 引用。
- **表格编号与引用:** `\caption{Sample data}` 生成标题, `\label{tab:sample}` 绑定编号, 正文用 `\ref{tab:sample}` 引用; 表格为严格的 2 行 2 列。
- **交叉引用机制:** 引用信息写入 .aux 文件, 需要编译两遍才能解析; latexmk 会自动重跑直到引用稳定, 因此最终 PDF 不会出现 “??”。
- **中文支持:** 模板为英文内容, pdflatex 可直接编译; 如文档中需要使用中文, 可安装 texlive-lang-chinese, 用 `\usepackage[UTF8]{ctex}` 配合 `latexmk -xelatex` 编译 (见 scripts/report-zh.tex)。

6 遇到的问题与解决

问题	原因	解决
新环境 git commit 报 “Please tell me who you are”	未配置 user.name / user.email	提交前执行 <code>git config user.name、git config user.email</code>
验证命令 <code>pdftotext: command not found</code>	pdftotext 属于 poppler-utils 软件包, 未安装	apt 安装 poppler-utils
交叉引用初次编译显示 “??”	引用解析依赖 .aux 文件, 需编译两遍	使用 latexmk, 其会自动重跑直至引用稳定

7 实验总结

通过本次实验, 我掌握了含空格文件名的安全处理方式 (`find -exec 传参 + cp --parents` 保留目录结构) 与目录/文件权限设置 (750/640); 熟练运用 `tail | awk | sort | uniq -c | sort | head` 管道完成日志统计, 并手算验证了统计结果; 理解了 Git 合并冲突的产生条件 (双方从同一基线修改同一行) 与标准解决流程 (编辑、add、commit); 掌握了 LaTeX 中公式、表格的自动编号与交叉引用 (label / ref / eqref), 以及 `latexmk -pdf -halt-on-error` 的命令行构建与验证方法。

附录：文件清单

<code>scripts/q01.sh</code>	第 1 题完整脚本
<code>scripts/q02.sh</code>	第 2 题完整脚本（含 <code>access.csv</code> 与 <code>analyze.sh</code> 的生成）
<code>scripts/q03.sh</code>	第 3 题完整脚本
<code>scripts/q04.sh</code>	第 4 题完整脚本（含依赖安装、构建、验证）
<code>scripts/report.tex</code>	第 4 题 <code>report.tex</code> （英文版）
<code>scripts/report-zh.tex</code>	第 4 题中文版 <code>report.tex</code> （需 <code>texlive-lang-chinese + xelatex</code> ）
<code>实验报告.tex</code>	本报告 LaTeX 源码（ <code>ctex + xelatex</code> 编译）
<code>实验报告.pdf</code>	编译生成的报告 PDF（8 页）